CODING AGENT
IMPLEMENTATION BRIEF

Heliocentric Solar-Power Swarm

A reproducible Python research simulator for heliocentric orbital dynamics, solar-energy collection, interplanetary beam transmission, thermal constraints, and constellation optimization.

Purpose
Give an autonomous coding agent a precise, staged specification for building the computational foundation of a publishable feasibility and optimization study.

Research framing
This project studies a distributed heliocentric solar-power swarm. It is not a claim that a complete Dyson sphere is currently buildable, nor a design for a literal solid shell around the Sun.

Document control	Value
Version	1.0
Prepared	11 November 2026
Primary implementation language	Python
First deliverable	One-satellite end-to-end simulator with verified physics interfaces
Intended use	Research code, reproducible figures, manuscript methods and results

Core rule: implement the simplest physically correct model first; validate it; then increase fidelity. Do not build a visually impressive simulator whose underlying physics cannot be checked.

Prepared as a hand-off specification for a coding agent. Candidate architectures and technology parameters are explicitly marked as exploratory unless stated otherwise.

1. Executive brief and research mission

Build an open, modular simulator that answers one central question: under what orbital, optical/RF, thermal, and receiver conditions can solar collectors in heliocentric orbit deliver useful electrical power to Earth? The simulator must be able to return a negative result. A physically infeasible architecture is a valid research finding if the limiting assumptions and feasibility boundary are quantified.

The codebase is a research instrument, not a science-fiction animation. Its primary output is a time series and uncertainty-bounded estimate of electrical power delivered to a specified Earth receiver network, with every conversion, loss, geometry, and availability condition visible.

Research objectives

ID	Objective	Required outputs
O1	Heliocentric orbital architecture	Satellite states, Earth range, line of sight, solar occultation, access windows, pointing geometry and stability indicators.
O2	Solar collection and thermal feasibility	Incident flux, PV output, waste heat, radiator area and temperature feasibility versus heliocentric distance.
O3	Interplanetary power-beam link	Diffraction pattern, capture fraction, pointing sensitivity, atmosphere and receiver conversion losses for optical and microwave cases.
O4	Constellation/system optimization	Average delivered power, availability, required area/aperture/mass and Pareto trade-offs.
O5	Comparative feasibility	Harmonized comparison against GEO/other SBSP baselines; identify conditions under which heliocentric deployment helps or hurts.

Primary research outputs

- A tested Python package and command-line interface that reproduces all headline simulations from versioned configuration files.
- A one-satellite end-to-end baseline, followed by multi-satellite cases.
- A set of physically interpretable plots: orbit and access, solar/thermal performance, beam capture, delivered-power time series, and feasibility/Pareto maps.
- A validation report documenting analytical limits, convergence, independent checks and model uncertainty.
- A manuscript-ready methods/results data package. No paper claim may be made unless it is traceable to a run and configuration.

The first manuscript should focus on orbital geometry + diffraction-limited transmission + thermal feasibility + constrained optimization. Coherent swarm beamforming, autonomous construction, detailed spacecraft hardware and economic modeling are later extensions.

2. Scope, design decisions, and non-goals

What is fixed for the first implementation

- The system is a distributed constellation of independently orbiting heliocentric collectors. Each satellite collects sunlight, converts it to electrical power, and transmits some fraction toward a terrestrial receiver.
- Python is the central implementation language. Core physical calculations must be callable as ordinary tested functions, not trapped inside notebooks.
- Start with one spacecraft and one receiver. Generalize to a constellation only after the end-to-end single-spacecraft calculation passes tests.
- Optical laser and microwave transmission are both modeled as alternative scenarios. Neither is preselected as the winning technology.
- Earth delivery is the key metric. Collected solar power or transmitter output alone is not a successful system result.
- A GEO solar-power system is a comparison baseline, not the assumed target architecture.

Exploratory options - do not treat as requirements

Option	Initial sweep	Status
Heliocentric radius	0.3, 0.5, 0.7, 1.0, 1.2 AU	Candidate values; expand only when useful.
Orbit shape	Circular first; then eccentric/inclined	Circular is the MVP baseline.
Constellation size	1, 10, 100, 1,000 satellites	Scale after single-satellite validation.
Microwave carrier	Example: 2.45 GHz	Illustrative only; make configurable.
Optical wavelength	Example: 1 micrometre	Illustrative only; make configurable.
Receiver radius	1, 5, 10 km	Scenario values, not design recommendations.
Relay architecture	Direct link first; relay as later extension	Do not assume relays are lossless.
Coherent distributed array	Not in MVP	Requires phase, timing and formation-control model.

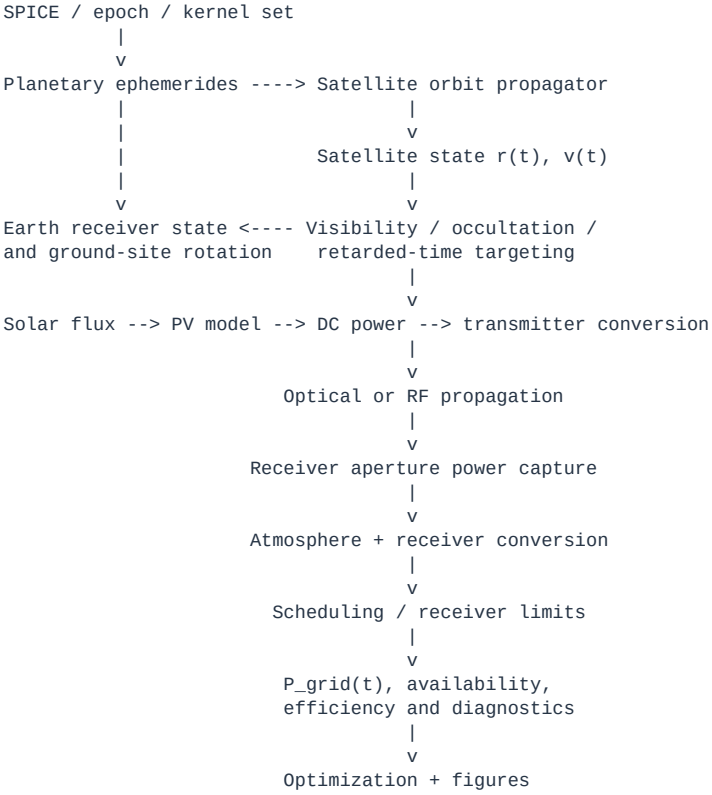
Explicit non-goals for the first paper

- Do not simulate a literal Dyson shell, stellar-scale construction, autonomous mining or self-replicating industry.
- Do not claim that a swarm of independent transmitters forms one coherent aperture. Independent transmitted powers add; coherent aperture synthesis requires phase-locked fields and a control model.
- Do not claim that optical or microwave transmission is feasible merely because the central beam intensity is high. Compute finite receiver capture and pointing losses.
- Do not present launch cost, levelized cost of energy or industrial-scale manufacturing claims as precise until a transparent mass and cost model exists.
- Do not model the Sun as a point source for passive solar concentration without stating the brightness/étendue limit. The initial project is active electrical conversion and beaming.

The project should be named and described as a heliocentric solar-power swarm feasibility simulator. 'Dyson swarm' can be used as the motivating concept, but the paper must define the modeled architecture precisely.

3. System architecture and data flow

The simulator should be built as a chain of independently testable models. Each stage accepts typed physical inputs and returns a structured result with units and diagnostics.



Required physical boundary

The power accounting boundary begins at incident sunlight on the collector and ends at electrical power accepted by the terrestrial grid interface. Every loss between those boundaries must be represented exactly once. Avoid double-counting transmitter efficiency, atmospheric attenuation or receiver conversion.

Recommended state/result objects

Object	Minimum fields
EphemerisState	epoch, frame, origin, position_m[3], velocity_m_s[3], source_kernel_ids
SatelliteState	epoch, position_m[3], velocity_m_s[3], mass_kg, attitude/normal vector, force-model flags
AccessResult	visible, range_m, elevation_rad, occulted_by, light_time_s, transmit_direction_unit
SolarPowerResult	flux_W_m2, incidence_cosine, illuminated, absorbed_W, pv_electric_W, pv_heat_W
BeamResult	wavelength_m, aperture_m, range_m, beam_model, capture_fraction, pointing_loss, field diagnostics
ThermalResult	waste_heat_W, radiator_area_m2, equilibrium_temperature_K, feasible, margin
LinkBudgetResult	power_at_transmitter_W, power_at_receiver_W, atmospheric_factor, rx_electric_W, grid_W
RunManifest	config_hash, git_commit, package_versions, kernel hashes, seed, frame, units, timestamps

Time and coordinate conventions

- Use SI units internally: metres, seconds, kilograms, watts, radians, kelvin. Convert astronomical units and degrees only at input/output boundaries.
- Use a documented inertial frame (e.g. J2000/ICRF) and a documented origin. Never subtract positions expressed in different frames or origins.
- Use SPICE ephemerides as the authoritative planetary-state source. Record kernel names, versions and checksums.

- Represent time explicitly, including the time scale. SPICE ephemeris time and UTC must not be mixed implicitly.
- Keep the Earth-fixed receiver coordinates separate from the inertial state. Apply an explicit Earth-orientation transform.

4. Software stack and repository specification

Use a small, stable core dependency set. Add specialist libraries only when the relevant work package is reached. Pin versions in the lock file and record the actual runtime versions in every simulation manifest.

Layer	Preferred tool	Use
Language / environment	Python 3.11+; uv or equivalent	Reproducible environment and package management.
Numerics	NumPy, SciPy	Vectorization, ODE integration, root finding, quadrature, optimization.
Units / astronomy	Astropy	Unit-aware interfaces, time and coordinate checks; convert to SI at model boundaries.
Ephemerides	SpiceyPy + NASA/JPL SPICE kernels	Planet and Sun states, frame transformations and reference ephemerides.
Orbit cross-check	REBOUND / REBOUNDx	Independent N-body comparison and later perturbation experiments.
Optical propagation	Analytical Airy/Gaussian models first; HCIPy or POPPY second	Wavefront propagation, Fresnel effects and aperture validation.
Microwave model	NumPy/SciPy array-factor and aperture-field model	Antenna pattern, finite receiver capture, phase/taper errors.
Weather / RF	ITU-R recommendations; ERA5/xarray optional	Atmospheric availability and RF attenuation scenarios.
Optimization	SciPy.optimize first; pymoo later	Single-objective baseline, then constrained multi-objective/Pareto search.
GPU	JAX or Numba optional	Only accelerate profiled batched sweeps; CPU implementation remains reference.
Data / plots	Pandas, PyArrow, Matplotlib; Plotly optional	CSV/Parquet outputs, scientific plots and exploratory views.
Quality	pytest, Ruff, mypy, pre-commit, CI	Automated tests, formatting, typing and reproducible builds.

Repository layout

```
heliocentric-swarm/
  README.md
  pyproject.toml
  uv.lock
  .gitignore
  configs/
    mvp_single_satellite.yaml
    optical_1au_sweep.yaml
    microwave_1au_sweep.yaml
    constellation_demo.yaml
    optimization.yaml
  src/helioswarm/
    __init__.py
    constants.py
    config.py
    ephemeris.py
    frames.py
    orbit.py
    srp.py
    ground.py
    visibility.py
    solar.py
    thermal.py
    aperture.py
    optical.py
    microwave.py
    atmosphere.py
    link_budget.py
    scheduler.py
    system.py
    metrics.py
    optimization.py
    plotting.py
    cli.py
  tests/
    unit/
    integration/
    regression/
  experiments/
    README.md
    notebooks/
  data/
    kernels/              # do not commit large kernels; document retrieval
    external/             # cached external data, with provenance
  runs/
    .gitkeep
  figures/
  paper/
    manuscript.tex
    references.bib
    methods_notes.md
  docs/
    model_assumptions.md
    validation_report.md
    data_provenance.md
```

Engineering rules

- Production logic belongs in src/; notebooks are for exploration and visualization only.
- Every public model function has type hints, docstrings, explicit units and a short reference to the governing equation or standard.
- Use dataclasses or validated Pydantic models for configuration and result records. Do not pass unstructured dictionaries through the whole codebase.
- Avoid global mutable state. Pass ephemeris providers, configurations and random generators explicitly.
- Separate deterministic physics from stochastic uncertainty sampling.
- Use a single configuration file to define every physical parameter used in a run.

5. Configuration contract and command-line interface

The agent must implement configuration validation before implementing complex physics. Invalid units, missing physical values or contradictory settings should fail early with readable errors.

Example configuration

```
project:
  name: "single_satellite_optical_baseline"
  seed: 20261111
  output_dir: "runs/single_satellite_optical_baseline"

time:
  start_utc: "2030-01-01T00:00:00"
  duration_days: 365.25
  output_step_s: 3600
  integrator_step_s: 300

frames:
  inertial: "J2000"
  origin: "SOLAR SYSTEM BARYCENTER"
  earth_fixed: "IAU_EARTH"

orbit:
  model: "prescribed_ephemeris_perturbed_test_particle"
  initial_epoch_utc: "2030-01-01T00:00:00"
  semimajor_axis_au: 1.0
  eccentricity: 0.0
  inclination_deg: 0.0
  raan_deg: 0.0
  arg_periapsis_deg: 0.0
  true_anomaly_deg: 30.0
  include_planetary_perturbations: true
  include_solar_radiation_pressure: true
  cr: 1.3
  area_to_mass_m2_kg: 0.02

collector:
  area_m2: 10000
  pv_efficiency_reference: 0.30
  reference_temperature_K: 300
  temperature_coefficient_per_K: -0.0004
  absorptivity: 0.90
  incidence_mode: "sun_tracking"

transmitter:
  mode: "optical"
  wavelength_m: 1.0e-6
  aperture_diameter_m: 10.0
  electrical_to_beam_efficiency: 0.50
  pointing_sigma_rad: 1.0e-9
  aperture_illumination: "uniform_circular"

receiver:
  sites:
    - name: "example_site"
      latitude_deg: 0.0
      longitude_deg: 0.0
      altitude_m: 0.0
  radius_m: 1000
  min_elevation_deg: 10
  conversion_efficiency: 0.40
  grid_efficiency: 0.97
  max_input_power_W: 1.0e12

thermal:
  emissivity: 0.90
  radiator_temperature_limit_K: 450
  radiating_sides: 2
  radiator_view_factor: 1.0

models:
  beam: "airy_far_field"
  atmosphere: "scenario"
  weather_scenario: "clear"
  light_time: true
  sun_occultation: true
```

This is a schema example, not a recommended engineering design. The agent must validate parameter ranges and document every default. Add schema versioning so old runs remain interpretable.

Required CLI

```
helioswarm validate-config configs/mvp_single_satellite.yaml
helioswarm run configs/mvp_single_satellite.yaml --out runs/mvp
helioswarm plot runs/mvp
helioswarm sweep configs/optical_1au_sweep.yaml --out runs/optical_sweep
helioswarm optimize configs/optimization.yaml --out runs/optimization
helioswarm validate-models --report docs/validation_report.md
```

Required run outputs

- config_resolved.yaml: fully resolved configuration including defaults.
- manifest.json: code commit, dependency versions, kernel identifiers/checksums, time scale, frame, random seed and run timestamp.
- timeseries.parquet: time-indexed states, access flags, power stages and thermal outputs.
- summary.json: mean/median/p05/p95 grid power, capacity factor, availability, outage durations and efficiencies.
- figures/: plots produced directly from the saved results.
- logs/: warnings, numerical integration diagnostics and failed/invalid time steps.

6. Work package 1 - ephemerides, orbital dynamics and access

Goal: compute physically consistent satellite and Earth-receiver geometry over time. Do not begin with thousands of satellites. First prove that one test spacecraft and one receiver can be propagated, targeted and checked.

6.1 Ephemeris and state initialization

1. Load the required SPICE leap-seconds, planetary ephemeris and planetary constants kernels. Record the exact kernel files and checksums.
2. Query Sun, Earth and relevant planets in one inertial frame and one origin. Convert all states to SI.
3. Construct the spacecraft initial state from classical orbital elements relative to the Sun at the selected epoch, then transform to the chosen inertial origin/frame.
4. Write round-trip tests: state to orbital elements and back; compare SPICE Earth-Sun range against a trusted reference.

6.2 Propagation hierarchy

Use a hierarchy rather than immediately committing to a single high-fidelity integrator:

Level	Model	Purpose
L0	Analytical two-body Kepler propagation	Fast baseline and unit tests; no planetary perturbations.
L1	Numerical test-particle integration with prescribed SPICE planetary ephemerides	Primary research propagation; planets remain tied to authoritative ephemerides.
L2	REBOUND N-body integration initialized from SPICE states	Independent comparison and selected long-duration/formation experiments.
L3	SRP, attitude-dependent area and selected non-gravitational forces	Sensitivity and robustness studies.

For L1 in a barycentric inertial frame, the gravitational acceleration is the sum of direct accelerations from the Sun and included planets. If a heliocentric non-inertial frame is used instead, include the correct indirect acceleration. Do not mix the two formulations.

```
a_grav(r,t) = sum_j mu_j * (r_j(t) - r) / norm(r_j(t) - r)^3

P_srp(r) = P_srp_1AU * (AU / r_sun)^2
a_srp = P_srp(r) * Cr * (A_over_m) * s_hat_away_from_sun

dr/dt = v
dv/dt = a_grav + a_srp + optional_forces
```

6.3 Solar radiation pressure

- Include a configurable reflectivity coefficient Cr and effective area-to-mass ratio A/m.
- The first implementation may use a simple radial SRP acceleration. A later model should account for panel attitude and projected area.
- Do not silently omit SRP for very large, low-mass collectors. Report the A/m range for which SRP materially changes the orbit.
- Poynting-Robertson drag and other small effects are optional sensitivity extensions, not MVP requirements.

6.4 Earth access and occultation

- Compute satellite-to-ground-site range, line-of-sight unit vector and local receiver elevation.
- Check Earth blockage using a ray/ellipsoid intersection, not only a spherical Earth approximation in final results.
- Check whether the Sun blocks the direct beam path. Include a configurable solar-limb margin for optical transmission.
- Include receiver minimum elevation and any Sun-avoidance angle required by the receiver architecture.
- For multiple sites, calculate access independently, then pass valid links to the scheduler.

7. Work package 1 continued - light-time targeting and ground geometry

7.1 Retarded-time targeting

At astronomical-unit distances, light travel time is several minutes. The beam must be aimed at the receiver's arrival-time position, not its emission-time position. The receiver site moves both with Earth's orbital motion and Earth's rotation.

```
Given satellite position r_sat(t_emit):

1. Set tau = norm(r_site(t_emit) - r_sat(t_emit)) / c
2. Repeat:
       r_site_arrival = inertial_ground_site(t_emit + tau)
       tau_new = norm(r_site_arrival - r_sat(t_emit)) / c
       stop when abs(tau_new - tau) < tolerance
       tau = tau_new
3. Aim along:
       u_tx = (r_site_arrival - r_sat(t_emit)) / norm(...)
4. Re-evaluate occultation and pointing at the converged geometry.
```

Use a robust root solver or fixed-point iteration with a convergence limit and diagnostic. Store both emission epoch and arrival epoch. The first version may neglect relativistic light-time corrections, but that omission must be stated.

7.2 Ground receiver coordinates

- Configuration uses geodetic latitude, longitude and altitude.
- Convert geodetic coordinates to Earth-fixed Cartesian coordinates using a documented reference ellipsoid.
- Rotate Earth-fixed coordinates into the inertial frame at the signal arrival epoch using an explicit Earth-orientation model.
- Use Astropy/IERS or a documented IAU Earth-orientation approximation. Do not assume a fixed ground station is inertially stationary.
- Test site position, local up vector, elevation angle and daily rotation against known cases.

7.3 Orbital outputs and diagnostics

Metric	Definition / use
Earth range	Distance from satellite to receiver at the relevant light-time geometry.
Access fraction	Fraction of simulated time meeting geometric and operational access constraints.
Occultation fraction	Fraction of time the beam is blocked by Earth or Sun.
Pointing angle	Required inertial beam direction and its time derivative.
Light time	Emission-to-arrival interval; report min/mean/max.
Range rate	Relative radial velocity; useful for pointing and link diagnostics.
Orbit drift	Difference from the reference two-body or SPICE-based trajectory.

7.4 Acceptance tests

- A circular two-body orbit preserves radius and orbital energy within the selected integrator tolerance.
- The orbital period matches Kepler's third law within a stated numerical tolerance.
- Earth-Sun distance agrees with SPICE at selected epochs within the ephemeris/time-conversion tolerance.
- A synthetic fixed geometry produces the analytically expected light-time to within numerical tolerance.
- A receiver site rotates by approximately one sidereal rotation per sidereal day in the inertial frame.
- A test beam whose segment intersects the Earth or Sun is correctly flagged as occulted.

8. Work package 2 - solar collection and thermal feasibility

Goal: calculate electrical power actually available to the transmitter, together with the heat that the collector and power electronics must reject.

8.1 Solar irradiance

```
S(r) = S_1AU * (AU / r_sun)^2

P_abs = S(r) * A_collector * absorptivity * cos(incidence)
P_PV  = P_abs * eta_PV(T)
Q_PV  = P_abs - P_PV - explicitly_modelled_reflected_or_transmitted_power
```

Use the accepted solar irradiance near 1 AU as a configurable physical constant (nominally about 1,361 W/m^2). Track panel incidence, occultation and attitude. A Sun-tracking collector is a useful optimistic bound; a fixed-attitude collector is a separate scenario.

8.2 Temperature-dependent conversion

Implement an explicit, configurable PV efficiency law. A linear local approximation is acceptable for the first sweep, but it must be labeled as a local approximation and bounded so it cannot produce efficiencies below zero or above the chosen physical limit. A later study should use technology-specific measured efficiency/temperature curves.

```
eta_PV(T) = clip(
    eta_ref * (1 + gamma_T * (T - T_ref)),
    eta_min,
    eta_max
)
```

8.3 Thermal balance

Do not use a generic 'number of radiating sides' multiplier without defining geometry. The simplest lower-bound radiator model is:

```
Q_waste = Q_PV + Q_transmitter + Q_power_electronics + Q_other

Q_radiated = epsilon * sigma * A_rad_effective
             * (T_rad^4 - T_space^4)

A_rad_effective = A_physical * emitting_side_factor * view_factor

Require Q_radiated >= Q_waste at the chosen radiator temperature.
```

The model must keep absorbed solar power, exported electrical power, transmitter output and waste heat in one energy balance. Reflected sunlight is not waste heat. If radiator temperature is solved rather than prescribed, use a bracketed root solver and report whether a solution exists below the maximum allowable temperature.

8.4 Required results

- Solar flux and collector power versus heliocentric distance.
- PV efficiency and operating temperature versus distance and technology scenario.
- Waste heat and radiator area versus distance and transmitter operating point.
- Net electrical power available to the transmitter after power-conditioning losses.
- Thermal feasibility flag and margin to temperature limit.

8.5 Acceptance tests

- At 1 AU and normal incidence, the model recovers the configured reference solar irradiance.
- At fixed collector area and efficiency, power scales as 1/r^2.
- Zero incidence cosine or an eclipse yields zero direct collected power.
- The thermal solver closes the energy balance to a defined tolerance.

- Radiator area scales inversely with emissivity and with the fourth power of the selected temperature in the simple limiting case.

9. Work package 3 - beam physics and diffraction

Goal: calculate what fraction of transmitted power reaches a finite receiver. Diffraction is a first-order design constraint, not a decorative visualization.

9.1 Start with analytical limits

For a uniformly illuminated circular aperture in the Fraunhofer regime, the angular first-null radius is approximately 1.22 lambda/D. At range R, the corresponding first-null radius is approximately 1.22 lambda R/D. This is a scale estimate, not the captured-power fraction.

```
theta_first_null ~= 1.22 * wavelength_m / aperture_diameter_m
radius_first_null ~= range_m * theta_first_null

Fresnel_number = aperture_diameter_m**2 / (wavelength_m * range_m)
```

Compute the Fresnel number and choose a propagation model accordingly. If the far-field assumption is not justified, use a finite-distance Fresnel or equivalent propagation calculation. The agent must not silently use a Fraunhofer Airy formula outside its validity regime.

9.2 Optical propagation

- MVP: normalized Airy pattern for a uniformly illuminated circular aperture; also include a Gaussian-beam approximation as a separate model, not as an interchangeable label.
- Compute the two-dimensional capture fraction for a finite circular receiver. Include off-axis pointing error by shifting the beam relative to the receiver.
- Include aperture illumination taper and wavefront error as later configurable extensions.
- Use HCIPy or POPPY to cross-check selected optical cases and to add Fresnel propagation, phase aberrations and wavefront effects.
- Record whether the result is scalar diffraction. Do not imply that a scalar model includes polarization, nonlinear optics or high-power thermal distortion.

9.3 Microwave propagation

- MVP: implement a circular aperture/array-factor model with a clearly stated aperture illumination and element-pattern assumption.
- Compute finite receiver capture, sidelobes, pointing loss and phase-error sensitivity.
- Treat a continuous filled aperture and a sparse phased array as different geometries.
- A constellation of independent RF transmitters is not automatically one coherent array. Coherent combining is a later module requiring phase, frequency, timing and relative-position control.

9.4 Finite receiver capture

The core metric is the fraction of transmitted beam power intercepted by the receiver:

```
eta_capture = integral_over_receiver_area I(x,y) dA
              / integral_over_all_transverse_plane I(x,y) dA
```

For a centered ideal Airy pattern, compare numerical integration against the known encircled-energy limit at the first dark ring. For offset receivers, integrate the displaced intensity distribution over the receiver disk. Use adaptive quadrature or a converged two-dimensional grid.

9.5 Pointing and control errors

- Represent pointing uncertainty as an angular distribution (e.g. isotropic Gaussian in two transverse axes) with configurable sigma.
- Distinguish deterministic pointing bias from random jitter.
- Report capture efficiency versus pointing sigma and receiver radius.
- For coherent arrays, add path-length/phase errors and show how phase errors degrade the array factor.
- Do not equate an achievable pointing specification with a demonstrated capability; treat it as an input assumption.

10. Work package 3 continued - link budget, atmosphere and receiver

10.1 End-to-end link budget

Use a single, auditable sequence of power stages. Each efficiency factor must have one owner and one definition.

```
P_dc_to_tx = P_PV * eta_power_conditioning
P_beam     = P_dc_to_tx * eta_electrical_to_beam
P_aperture = P_beam * eta_transmitter_optics_or_rf
P_atm      = P_aperture * eta_atmosphere
P_captured = P_atm * eta_capture
P_receiver = P_captured * eta_receiver_conversion
P_grid     = P_receiver * eta_grid_conditioning
```

If an aperture model already returns the power distribution normalized to transmitted beam power, do not multiply by an additional 'aperture efficiency' unless that factor represents a separate physical loss. Name factors precisely.

10.2 Optical atmosphere

- For the MVP, use explicit clear/cloudy scenario factors and an availability mask. Do not infer a precise optical transmission value from total cloud fraction alone.
- Later, use site-specific cloud and atmospheric data to estimate operational availability. Cloud obstruction can be a hard outage for a ground optical receiver.
- Add turbulence/seeing only if the receiver architecture and wavefront correction assumptions are specified.
- Keep atmospheric transmission separate from diffraction capture and receiver conversion.

10.3 Microwave atmosphere

- Use current ITU-R recommendations for Earth-space gaseous, precipitation and cloud attenuation where their frequency and geometry domains apply.
- Record recommendation number and version in the run manifest.
- Include polarization, frequency and elevation-angle assumptions where relevant.
- Do not apply terrestrial RF attenuation formulas to the interplanetary vacuum segment. The atmosphere is only a short terminal segment near Earth.

10.4 Receiver and safety constraints

- Define receiver aperture/area, conversion efficiency, maximum input power and thermal rejection capability.
- Add a maximum allowable incident power density and beam-abort condition as explicit configurable constraints. These are engineering constraints, not optional post-processing.
- A link is unavailable if pointing uncertainty, site elevation, weather, occultation or receiver state violates the configured conditions.
- For multiple transmitters, enforce receiver power limits and prevent double-counting the same receiver capacity.
- Treat optical and microwave receivers as different technology cases; do not assign them the same conversion or safety assumptions.

10.5 Metrics

Metric	Report
End-to-end efficiency	P_grid / incident solar power intercepted by collector.
Beam capture	Fraction of emitted beam power intercepted by receiver.
Availability	Fraction of time the link satisfies all geometry, weather, pointing and receiver conditions.
Capacity factor	Mean delivered power divided by the specified rated/target power; define denominator clearly.
Outage statistics	Number, duration distribution and longest outage.
Delivered energy	Integral of P_grid over the simulation interval.
Power density	Receiver incident W/m^2 and any modeled beam-path safety constraints.

11. Work package 4 - integrate one satellite, then scale to a swarm

11.1 Mandatory vertical slice

Before any constellation work, produce a complete end-to-end run for one satellite and one receiver. This is the project's first major milestone.

1. Propagate one satellite on a documented heliocentric orbit.
2. Calculate solar flux, PV output and waste heat.
3. Calculate the transmitter electrical input and emitted beam power.
4. Compute retarded-time receiver geometry and occultation.
5. Compute diffraction pattern, pointing offset and receiver capture.
6. Apply atmosphere, receiver conversion and grid conditioning.
7. Save P_grid(t), all intermediate power stages, thermal feasibility and link diagnostics.
8. Generate a one-page run report with the configuration and plots.

The first vertical slice may use simplified atmospheric conditions and a simple circular-aperture beam, but it must preserve correct units, energy accounting, time geometry and receiver capture.

11.2 Constellation scaling

- Represent each satellite as an independent state and technology configuration. Vectorize common calculations where practical.
- Start with identical satellites on one orbital ring and evenly spaced phases; compare against randomized phases.
- Add multiple rings, inclinations and eccentricities only after the one-ring case is validated.
- For each epoch, create a list of feasible satellite-to-receiver links, including power capability and link constraints.
- Schedule power among receivers and satellites subject to receiver capacity, beam exclusivity, safety interlocks and any transmitter duty-cycle constraint.
- Use an event-driven or time-step scheduler that is deterministic and tested. Do not simply sum every possible link if a satellite or receiver cannot serve them simultaneously.

11.3 Independent versus coherent transmission

Architecture	Model treatment	When to implement
Independent satellites	Sum received electrical power from independent links, subject to receiver capacity and scheduling.	MVP constellation.
Coherent distributed aperture	Complex-field sum with relative phase, delay, path error, frequency and timing control.	Later research extension only.
Relay chain	Each relay has capture, conversion, thermal, pointing and retransmission losses.	Later architecture comparison.

11.4 Required system outputs

- P_grid(t) for each receiver and the entire network.
- Time-resolved active satellite count and valid link count.
- Mean, median, percentile and peak power.
- Availability and outage duration statistics.
- Total collector area, radiator area, transmitter aperture and mass proxy.
- Net delivered power per collector area and per assumed deployed mass.

- A clear breakdown of why power is unavailable: geometry, occultation, weather, pointing, thermal limit, receiver saturation or scheduler decision.

12. Work package 5 - optimization and uncertainty

Do not run a genetic algorithm before the forward simulator is correct. Optimization can efficiently find the wrong answer if the physical model is wrong.

12.1 Optimization variables

Variable group	Candidate variables
Orbit	Semimajor axis, eccentricity, inclination, orbital phase and ring count.
Constellation	Satellite count, phase distribution, collector area per satellite.
Transmitter	Technology, wavelength/frequency, aperture diameter, beam taper and pointing sigma.
Receiver	Receiver radius/area, site count, site locations and receiver capacity.
Thermal	Radiator temperature, emissivity, view factor and radiator specific mass.
Operations	Weather margin, availability requirement, scheduling policy and minimum elevation.

12.2 Objectives and constraints

- Maximize time-averaged grid power.
- Minimize total system mass or a transparent mass proxy.
- Maximize availability above a specified minimum power threshold.
- Minimize collector area or receiver infrastructure where appropriate.
- Constrain maximum temperature, minimum availability, receiver capacity, pointing error, aperture size and allowed beam irradiance.
- Report Pareto fronts rather than collapsing all objectives into one arbitrary weighted score.

12.3 Suggested sequence

1. Use parameter sweeps and SciPy optimization for single-objective sanity checks.
2. Use a small number of variables and deterministic runs to verify the optimizer can recover known synthetic optima.
3. Only then use pymoo (e.g. NSGA-II) for multi-objective design-space exploration.
4. Repeat optimization with multiple seeds and convergence checks. Preserve the full Pareto population and run manifest.
5. Re-evaluate every selected candidate with the highest-fidelity forward model, not only the optimizer's cheapest approximation.

12.4 Uncertainty and sensitivity

- Separate epistemic uncertainty (unknown technology performance) from aleatory variability (weather, jitter and operational randomness).
- Use seeded Monte Carlo sampling for uncertain PV efficiency, transmitter efficiency, pointing jitter, phase error, atmospheric availability and radiator specific mass.
- Use Sobol or another variance-based sensitivity method after the model is stable.
- Report confidence/credible intervals or scenario ranges, not only a single best-case curve.
- Keep optimistic, nominal and conservative scenarios explicit in configuration.

12.5 Economics - optional later module

A cost-per-kWh model is useful only after mass, deployment and lifetime assumptions are transparent. Add it as a separate parameterized module, not as a core physical truth. Inputs may include manufacturing cost per kilogram, transport cost, launch/deployment assumptions, lifetime, replacement rate, operations cost and discount rate. Compare scenarios and uncertainty; do not present a single speculative LCOE as a robust prediction.

13. Verification, validation and scientific quality gates

Every work package has to pass its own acceptance tests before integration. The validation report is a required deliverable, not an optional appendix.

13.1 Unit tests

Module	Minimum unit tests
constants / units	Dimensional consistency; AU, solar flux, c, sigma and gravitational parameters.
frames / ephemeris	Known frame transforms; consistent origin; kernel coverage and time-scale checks.
orbit	Kepler period, energy/angular momentum conservation in two-body mode, convergence with step size.
SRP	Correct inverse-square scaling and direction away from Sun for the simple model.
ground / visibility	Known station coordinates, Earth rotation, elevation and occultation edge cases.
solar	1/r^2 flux scaling, incidence and eclipse limits, efficiency bounds.
thermal	Energy closure and analytic radiative-equilibrium limiting cases.
beam	Airy first-null scaling, normalized total power, capture convergence, pointing-offset monotonicity.
link budget	Power never increases through passive stages; all efficiencies lie in [0,1].
scheduler	No receiver over-capacity, no double allocation and deterministic scheduling.
optimization	Synthetic known optimum, bounds and repeatability.

13.2 Integration tests

- One-satellite 1-AU baseline runs from config to saved timeseries and summary without manual notebook steps.
- A deliberately impossible beam configuration yields near-zero receiver capture rather than an unphysical high delivered power.
- An occulted or unavailable link delivers zero power.
- A receiver capacity limit clips or schedules incoming power correctly.
- A higher collector area increases available PV power linearly when all other assumptions are fixed.
- A smaller wavelength or larger aperture improves the ideal diffraction scale in the expected direction.
- Increasing pointing uncertainty cannot improve expected capture in the centered symmetric baseline.

13.3 Independent checks

- Compare selected orbital trajectories against REBOUND and/or GMAT/Orekit. State why any discrepancy exists (force model, frame, time scale or integrator).
- Compare planetary positions and Sun-Earth range against SPICE/JPL ephemerides.
- Compare Airy patterns and encircled energy against analytical solutions.
- Cross-check selected optical propagation cases with HCIPy or POPPY.
- Cross-check RF attenuation calculations against the applicable current ITU-R recommendation or a validated implementation.
- Run time-step and spatial-grid convergence tests for all numerical integrations and capture calculations.

13.4 Research integrity rules

- Never fabricate numerical results, citations, validation outcomes or software capabilities.
- Every figure must be generated from saved run data and state its configuration.
- Every parameter must be labeled measured, sourced, assumed or swept.
- If a model is only an idealized upper bound, label it as such in the figure and manuscript.
- Record negative and failed runs; do not silently discard inconvenient results.

- Keep raw input data, derived outputs and plotting scripts distinct.

14. Reproducibility, data provenance and performance

14.1 Reproducible runs

- Every run has a unique output directory and a resolved configuration snapshot.
- Record Git commit hash, Python version, dependency versions, operating system, CPU/GPU device, random seed and simulation start/end time.
- Record SPICE kernel names, versions, source URLs and SHA-256 checksums. Do not commit large kernel files unless licensing and repository policy permit.
- Record external weather-data product, retrieval date, variables, spatial/temporal resolution and license/attribution.
- All random operations receive an explicit NumPy Generator seeded from configuration.

14.2 Data formats

- Use Parquet for large time series; CSV may be exported for inspection.
- Use JSON for compact summaries and manifests.
- Use NetCDF/Zarr only when working directly with gridded weather or field data.
- Use FITS only where a specialist astronomy/optics library naturally produces it.
- Do not store large arrays as giant JSON lists.

14.3 Performance strategy

1. Profile the single-satellite vertical slice before optimizing.
2. Vectorize satellite/time calculations and cache ephemeris samples when valid.
3. Use coarse-to-fine sweeps: cheap analytical beam model for screening, high-fidelity propagation for finalists.
4. Parallelize independent scenarios and Monte Carlo samples before rewriting physics in GPU code.
5. Add JAX/Numba only if profiling identifies a clear bottleneck and the accelerated version passes the same reference tests.
6. Maintain a CPU reference implementation so GPU results can be cross-checked.

14.4 Documentation

- README: installation, quick-start run, expected outputs and project status.
- model_assumptions.md: every equation, assumption, validity range and simplification.
- data_provenance.md: kernel and external-data provenance.
- validation_report.md: tests, independent comparisons, tolerances and unresolved discrepancies.
- experiment log: question, config, code commit, result location and interpretation for each scientific experiment.

15. Implementation milestones and acceptance gates

Milestone	Agent work	Exit condition
M0 - Scaffold	Create repo, environment, package layout, config schema, CLI shell, CI and tests.	Clean install; lint/type/test commands pass; config validates.
M1 - Analytic orbit	Implement two-body orbital elements, state propagation and unit tests.	Kepler period and invariants pass documented tolerances.
M2 - Ephemeris geometry	Load SPICE kernels; get Sun/Earth/planet states; transform frames; initialize satellite.	Known ephemeris checks pass; state metadata recorded.
M3 - Access and light time	Ground station transform, line of sight, Earth/Sun occultation, retarded-time targeting.	Synthetic geometry and Earth-rotation tests pass.
M4 - Solar and thermal	PV output, temperature law, heat balance, radiator sizing.	Energy closure and limiting-case tests pass.
M5 - Beam physics	Airy/Gaussian baseline, finite receiver integration, pointing errors; RF model.	Analytical diffraction and capture tests pass.
M6 - End-to-end single satellite	Join orbit, collection, thermal, beam, atmosphere, receiver and grid budget.	One config produces P_grid(t), manifest, summary and plots.
M7 - Constellation	Multiple satellites, rings/phases, receiver scheduling and capacity.	No double counting; deterministic capacity-constrained results.
M8 - Optimization/uncertainty	Sweeps, constrained optimization, Pareto results and sensitivity.	Synthetic optimization tests and repeatability pass.
M9 - Paper package	Freeze configs, produce figures/tables, write methods/validation notes.	Every paper result traceable to code, config and run data.

Priority order

Do not parallelize all work packages immediately. The critical path is M0 -> M1 -> M2 -> M3 -> M4/M5 -> M6. Constellation optimization is downstream of a validated single-satellite model. A coding agent may prepare interfaces for later modules, but should not fill the repository with unvalidated stubs.

Minimum publishable computational result

- A validated orbital-access model for a small number of heliocentric architectures.
- An end-to-end optical and microwave link budget with finite receiver capture and pointing uncertainty.
- Thermal feasibility constraints for the assumed collector/transmitter operating points.
- A small, defensible constellation comparison with availability and delivered-power metrics.
- A transparent comparison against a GEO baseline under harmonized assumptions.
- A documented parameter sensitivity analysis and reproducible release.

16. Manuscript outputs and figure plan

Working title: **Orbital Dynamics and Wireless Power Transmission Constraints of a Heliocentric Solar-Power Swarm: A Computational Feasibility and Optimization Study**. The title should be revised after results establish the strongest contribution.

Manuscript structure

Section	Content generated by the project
1. Introduction	Prior SBSP and Dyson-swarm concepts; precise research gap; contributions and hypotheses.
2. Architecture	System diagram, orbit families, collector/transmitter/receiver assumptions.
3. Physical models	Orbital equations, SRP, light time, solar collection, thermal balance, diffraction and link budget.
4. Numerical methods	Ephemerides, integrators, beam propagation, scheduling, optimization and uncertainty.
5. Verification	Analytical limits, convergence, independent software checks and energy closure.
6. Results	Orbit access, power/thermal curves, diffraction capture, time-series delivery and Pareto fronts.
7. Discussion	Dominant bottlenecks, limitations, technology assumptions, GEO comparison and future work.
8. Conclusion	Quantified feasibility boundary and conditions required for improved performance.

Figure plan

Figure	Content	Source module
1	System architecture diagram with direct beam and optional relay path.	paper/diagram; configuration schema
2	Heliocentric trajectories and satellite-Earth range/visibility versus time.	orbit.py, visibility.py
3	Solar irradiance, PV output and radiator requirement versus orbital radius.	solar.py, thermal.py
4	Beam intensity maps and receiver capture versus aperture, wavelength and pointing.	optical.py, microwave.py
5	End-to-end power budget and delivered-power time series.	link_budget.py, system.py
6	Constellation availability and power versus satellite count and orbital phase.	scheduler.py, metrics.py
7	Pareto frontier and feasible/infeasible design regions.	optimization.py
S1+	Convergence, validation, uncertainty and parameter tables.	tests/ and validation report

Comparative baseline

Use a GEO SBSP reference as a benchmark, but harmonize assumptions before comparing: delivered power, PV efficiency, transmitter efficiency, receiver conversion, system lifetime, collector technology, thermal treatment and what is included in mass/cost. NASA's 2024 SBSP assessment is a useful source for representative GEO architectures and for understanding the scale of lifecycle assumptions; it is not a heliocentric design model.

The manuscript must not claim that the heliocentric architecture is superior unless the same system boundary and comparable technology assumptions support that conclusion. A negative result or a narrow feasibility envelope is publishable if it is robust and well explained.

17. Main technical risks and how the agent should handle them

Risk	Why it matters	Required mitigation
Frame/time inconsistency	Can create large position and pointing errors.	One frame/origin/time contract; explicit conversions; tests against SPICE.
Diffraction model outside validity	Can under/overestimate beam footprint.	Calculate Fresnel number; use Fresnel propagation where required.
Pointing treated as perfect	Can dominate capture for narrow beams.	Model bias and jitter; report capture distributions.
Coherence assumed without control	Incorrectly turns a swarm into a giant aperture.	Independent powers in MVP; coherent complex-field model only as separate extension.
Thermal losses omitted	Can make near-Sun operation appear unrealistically attractive.	Close energy balance; radiator feasibility and mass proxy.
Atmosphere over-simplified	Ground optical availability may be overestimated.	Scenario masks first; site-specific weather/statistical model later.
Receiver double-counting	Multiple satellites may exceed receiver capacity.	Explicit scheduler and receiver power/thermal limits.
Optimization of a flawed model	Produces precise but invalid optima.	Validation gates before optimization; high-fidelity reevaluation.
Economic overclaim	Mass and launch assumptions are uncertain.	Keep economics optional and parameterized; publish sensitivity ranges.
Overengineering early	Delays first meaningful result.	One-satellite vertical slice; add complexity only when it changes conclusions.

Agent behavior rules

- Before coding a new module, inspect existing interfaces and tests. Do not create duplicate implementations.
- For each milestone, produce a short implementation note: files changed, equations implemented, assumptions, tests run, unresolved issues.
- Never silently replace a requested physical model with a convenient approximation. If using an approximation, name it, document its validity and add a comparison path.
- If a dependency is unavailable or incompatible, report the issue and propose the smallest supported alternative; do not invent APIs.
- Do not download or commit large kernels/data without documenting provenance, licensing and size.
- Do not write manuscript conclusions before the validated runs exist.

18. Copy-paste kickoff prompt for the coding agent

The following prompt is designed to be given directly to an autonomous coding agent. It tells the agent what to build first and how to work without jumping ahead.

```
You are the lead research-software engineer for a computational physics project:
"Heliocentric Solar-Power Swarm Feasibility Simulator".

MISSION
Build a reproducible Python simulator that evaluates whether solar-collecting
spacecraft in heliocentric orbit can deliver electrical power to specified
Earth receivers through optical or microwave beams. This is a research
feasibility model, not a claim that a full Dyson sphere is buildable.

READ THIS SPECIFICATION FIRST
Treat this PDF as the project requirements document. Preserve the distinction
between fixed scope and exploratory options. Candidate orbit radii, satellite
counts, wavelengths and receiver sizes are scenario inputs, not established
design decisions.

WORKING RULES
1. Implement in milestones. Do not jump to optimization or a huge constellation.
2. Start with a clean repository, environment, config schema, CLI and tests.
3. Build one complete single-satellite end-to-end vertical slice before scaling.
4. Use SI units internally and explicit coordinate frame, origin and time scale.
5. Use SPICE/JPL ephemerides as the authoritative planetary reference.
6. Use analytical two-body dynamics as a baseline; add prescribed-ephemeris
   perturbations and SRP only with tests.
7. Model finite receiver capture, pointing errors and light-time targeting.
8. Maintain a closed power/thermal budget. Never double-count efficiencies.
9. Independent satellites add received powers; do not assume coherent aperture
   synthesis unless phase/timing/formation control is explicitly modeled.
10. Every output must be reproducible from a versioned config and run manifest.
11. Never fabricate results, citations, data, API behavior or validation.
12. Keep notebooks exploratory; production logic belongs in src/.

FIRST TASK: MILESTONE M0
Inspect the current workspace. If it is empty, create the repository layout
specified in this PDF. Add:
- pyproject.toml and a locked Python environment;
- src package and CLI entry point;
- config schema and one minimal YAML config;
- pytest, Ruff and type-check configuration;
- README with installation and one placeholder quick-start;
- CI workflow if the environment supports it.

Do not implement complex physics yet. Make config validation and a
"validate-config" CLI command work. Add tests and report exactly what passes.

THEN MILESTONE M1
Implement constants and an analytical two-body orbit module:
- convert classical orbital elements to position/velocity;
- propagate a Keplerian orbit;
- return states in SI units;
- test Kepler's third law, period and conservation invariants;
- document frame/origin assumptions.

MILESTONE M2
Add SPICE ephemeris loading and frame/time handling. Record kernel provenance.
Cross-check Sun-Earth distance and selected planetary states. Add a satellite
test-particle propagator with prescribed planetary ephemerides. Use REBOUND
as an independent cross-check where practical.

MILESTONE M3
Implement Earth ground-site coordinates, Earth rotation, line of sight,
Earth/Sun occultation and retarded-time targeting. Store emission and arrival
epochs. Test against synthetic geometries.

MILESTONE M4
Implement solar flux, PV conversion, temperature dependence and thermal
radiator balance. Add energy-closure tests.

MILESTONE M5
Implement analytical Airy/Gaussian beam models, finite receiver integration,
pointing bias/jitter and an RF aperture/array-factor model. Determine whether
far-field assumptions apply; use Fresnel propagation when necessary. Cross-
check selected optical cases with HCIPy or POPPY.
```

```
MILESTONE M6
Integrate one satellite and one receiver end to end. The run must write:
resolved config, manifest, timeseries, summary metrics and figures. It must
produce P_grid(t) and every intermediate power stage.

MILESTONE M7-M9
Only after M6 validation: add constellation scheduling, optimization,
uncertainty/sensitivity, GEO comparison and manuscript-ready outputs.

DELIVERABLE AFTER EACH MILESTONE
Return:
A. concise summary of implementation;
B. list of files added/changed;
C. exact commands run;
D. tests and results;
E. assumptions and equations implemented;
F. known limitations and next milestone.

STOP CONDITIONS
Stop and ask for clarification only if a genuinely blocking physical choice
cannot be represented as a configurable scenario. Otherwise use the explicit
MVP defaults in the spec, document them and continue. Do not spend time
polishing plots before the physical model and tests pass.
```

The first agent session should end at M0, not attempt to complete the entire simulator in one pass. Review the repository scaffold and tests before authorizing the next milestone.

19. Technical references and documentation

The coding agent should consult primary documentation and current standards. Confirm the current version and domain of applicability before implementing a standard or depending on a library API.

Topic	Reference / use
NASA/JPL SPICE	NAIF SPICE Toolkit and documentation: naif.jpl.nasa.gov/naif/toolkit.html. Use for ephemerides, time and frames.
SpiceyPy	Python wrapper documentation: spiceypy.readthedocs.io.
REBOUNDx	REBOUNDx documentation: reboundx.readthedocs.io. Additional forces for N-body integrations.
HCIPy	HCIPy documentation: docs.hcipy.org. Optical wavefront and propagation tools.
POPPY	POPPY documentation: poppy-optics.readthedocs.io. Optical propagation and Fresnel options; note documented limitations.
pymoo	pymoo multi-objective optimization guide: pymoo.org/getting_started.
ITU-R P.618	Current in-force Earth-space propagation recommendation: ITU-R P.618. Check applicable frequency and geometry limits.
ITU-R P.676	Atmospheric-gas attenuation recommendation: ITU-R P.676.
ERA5	Copernicus ERA5 hourly single-level data and product guide: ERA5 dataset. Record variables, resolution and provenance.
NASA SBSP assessment	NASA Office of Technology, Policy and Strategy, Space-Based Solar Power (2024): NASA report PDF. Useful GEO reference designs and lifecycle assumptions.

How to use the references

- Prefer official documentation and standards for software behavior and propagation methods.
- Use peer-reviewed papers for scientific claims and literature positioning; software documentation is not a substitute for a literature review.
- Cite the exact standard revision and exact software version used in the manuscript.
- Check license and redistribution terms before bundling kernels, weather data or external datasets.
- The NASA GEO assessment is a comparator and context source; its system assumptions must not be silently transplanted into the heliocentric model.

Final definition of done: another researcher can clone the repository, install the locked environment, obtain the documented ephemeris kernels, run the supplied configurations, regenerate every main figure, inspect all assumptions and reproduce the reported power-delivery metrics without relying on undocumented manual steps.